TraceAdvisor: Pre-Execution Resource Recommendations for Recurring Cluster Workloads

Shivani Jariwala, Meghana Koti, Sonali Lonkar, and Sofia Silva
Computer Engineering Department
San José State University (SJSU)
San José, CA, USA

***Abstract*—Cloud and AI clusters often experience inefficient resource utilization because users must estimate CPU and memory requirements before execution. To avoid under-provisioning, which can lead to out-of-memory events, evictions, or job failures, users tend to request more resources than their workloads need. This results in over-provisioning and unused resource capacity in shared clusters. Existing approaches often operate at runtime, require scheduler modifications, or depend on online experimentation rather than providing recommendations before job submission. This project presents TraceAdvisor, an offline, trace-driven system that generates pre-execution CPU and memory recommendations for recurring cluster workloads. The framework uses historical execution data to identify recurring workloads, reconstruct execution histories, and analyze resource usage patterns. TraceAdvisor compares original resource requests with statistical baselines, machine learning predictions, and hybrid recommendation policies. Historical statistics, including mean usage and percentiles such as P90, P95, and P99, account for variations in resource demand. Hybrid policies combine machine learning predictions with historical safeguards to balance efficiency and reliability. TraceAdvisor uses Google ClusterData2019 as its primary dataset and Alibaba GPU 2020 as a second production dataset covering recurring machine learning workloads. Dataset-specific processing and evaluation account for differences in workload structure and CPU and memory measurements. Chronological trace replay uses metrics such as violation rate, coverage, slack, and waste to examine trade-offs between over-provisioning and under-provisioning. The system supports resource allocation decisions before execution without requiring scheduler changes.**

***Index Terms*—Cluster workloads, CPU and memory recommendations, resource allocation, production traces, machine learning**

CONTENTS

I Project Overview 3
- I-A Introduction . . . 3
- I-B Problem Statement . . . 3
- I-C Literature Review . . . 3
  - I-C1 Over-Provisioning and Resource Allocation . . . 3
  - I-C2 Datasets and Trace Foundations . . . 3
  - I-C3 Existing Resource Recommendation Approaches . . . 4
  - I-C4 Research Gap . . . 4
- I-D Project Objectives and Scope . . . 4
- I-E Project Contributions . . . 5

II Project Architecture 6
- II-A Introduction and System Overview . . . 6
- II-B Architecture Subsystems . . . 6
  - II-B1 Data Ingestion and Dataset Adapters . . . 6
  - II-B2 Execution Reconstruction and Occurrence Modeling . . . 6
  - II-B3 Historical Features and Time-Ordered Data . . . 6
  - II-B4 Recommendation Engine . . . 6
  - II-B5 Shared Metrics and Evaluation Framework . . . 6
  - II-B6 Artifact Repository and Presentation Layer . . . 6

III Technology Descriptions 6
- III-A Data Processing and Analysis Stack . . . 6
- III-B Machine Learning Libraries and Models . . . 6
- III-C Dataset and Trace Sources . . . 6
  - III-C1 Primary Dataset: Google ClusterData 2019 . . . 6
  - III-C2 Validation Datasets: Azure Resource Central, Alibaba, ModServe, and Splitwise . . . 6
- III-D Dataset Adapters and Unit Normalization . . . 6
- III-E Artifact Storage, Reproducibility, and Presentation Technologies . . . 6

IV Project Design 6
- IV-A Data Pipeline Design . . . 6
- IV-B Recommendation Model Design . . . 6
  - IV-B1 Historical Percentile Baselines . . . 6
  - IV-B2 Machine Learning Forecasting . . . 6
  - IV-B3 Hybrid Risk-Aware Layer . . . 6
- IV-C Replay and Evaluation Framework Design . . . 6
- IV-D Artifact and Presentation Layer Design . . . 6

V Project Implementation 6
- V-A Data Pipeline . . . 6
  - V-A1 Ingestion, Unit Normalization, and Dataset Adapters . . . 6
  - V-A2 Execution Reconstruction and Dataset-Specific Temporal Features . . . 6
  - V-A3 Peak Usage and Resource Metrics . . . 6
- V-B Baseline Recommenders . . . 6
  - V-B1 Request-As-Is . . . 6
  - V-B2 Mean and Percentile Recommendations . . . 6
  - V-B3 Cold-Start Handling and Fallbacks . . . 6
- V-C Machine Learning Recommender . . . 6
  - V-C1 Feature and Target Preparation . . . 6
  - V-C2 Model Training and Selection . . . 6
  - V-C3 Validation-Based Model Comparison . . . 6
- V-D Hybrid Risk-Aware Recommender . . . 6
- V-E Shared Metrics and Evaluation Framework . . . 6
- V-F Artifact Storage and Developer-Facing Outputs . . . 6

VI Testing and Verification 6
VI-A Data and Pipeline Correctness Checks . . . 6
VI-B Temporal Leakage and Split Integrity . . . 6
VI-C Model and Baseline Validation . . . 6
VI-D Replay Evaluation Metrics . . . 6
VI-D1 Violation and Utilization . . . 6
VI-D2 Resource Waste . . . 6
VI-D3 Valid-Row Denominators and Missing-Value Handling . . . 6

VII Performance, Results, and Discussion 6
VII-A Google ClusterData 2019 Results . . . 6
VII-A1 Baseline Recommendation Results . . . 6
VII-A2 ML and Hybrid Recommendation Results . . . 6
VII-A3 Feature Importance and Explainability . . . 6
VII-B Cross-Platform Validation Results . . . 6
VII-C Discussion of Findings . . . 6

VIII Operations, Reproducibility, and Future Deployment 6
VIII-A Offline Artifact Workflow . . . 6
VIII-B Reproducibility and Repository Layout . . . 6
VIII-C Interactive Developer Dashboard (Planned) . . . 6

IX Summary, Conclusions, and Recommendations 6
IX-A Summary . . . 6
IX-B Conclusions . . . 6
IX-C Limitations and Recommendations for Further Research . . . 6

References 6

I. Project Overview

A. Introduction

Modern cloud and AI clusters often experience inefficient resource utilization because users must estimate CPU and memory requirements before execution. To avoid under-provisioning, which can lead to out-of-memory (OOM) events, eviction, or job failure, users tend to request more resources than their workloads actually need. This results in over-provisioning and unused resource capacity in shared clusters [1].

Production workload traces provide historical resource usage information that can help improve future allocation decisions. Studies such as Resource Central demonstrate that workload behavior can remain consistent across executions, making historical data useful for resource prediction [2]. Similarly, Accordia shows that recurring workloads can be optimized using information from previous executions [3]. These findings support the use of historical workload data to guide resource allocation decisions.

TraceAdvisor addresses resource allocation inefficiencies by providing CPU and memory recommendations before job submission. The framework uses historical execution data to identify recurring workloads, analyze resource usage patterns, and generate recommendations that account for higher resource demands. By providing guidance before execution rather than relying on runtime adjustments, TraceAdvisor aims to reduce unnecessary resource allocation while maintaining workload reliability.

B. Problem Statement

Existing resource management approaches attempt to address inefficient resource allocation through runtime optimization, provider-side scheduling, and configuration recommendations. For example, Flex proposes a runtime resource manager that uses oversubscription to improve resource utilization [1]. However, this approach operates after job submission and does not provide users with resource recommendations before execution. Similarly, QoS-Based Pricing and Scheduling focuses on provider-side pricing and scheduling rather than resource estimation before job submission [4].

Configuration recommendation systems also use historical workload behavior to improve resource allocation. Accordia, for example, optimizes cloud configurations for recurring batch workloads using Gaussian Process Upper Confidence Bound (GP-UCB) [3]. However, this approach relies on configuration search and online experimentation rather than directly providing resource recommendations from historical execution data. Although existing studies demonstrate that historical workload data can support resource prediction and optimization, many approaches operate at runtime, require scheduler integration, or depend on online experimentation. As a result, there remains a need for a lightweight, offline system that uses historical data from recurring workloads to provide CPU and memory recommendations before job submission without requiring scheduler modifications or online profiling.

C. Literature Review

1) Over-Provisioning and Resource Allocation: Multiple trace studies show that users frequently request more resources than their workloads actually consume, leading to inefficient resource utilization. For example, Flex analyzes Google cluster traces and identifies significant differences between requested and actual resource usage [1]. The study proposes a runtime resource manager that uses oversubscription to improve utilization. However, this approach operates after job submission and does not provide users with resource recommendations before execution.

Similarly, Google Cluster Deep Dive analyzes ClusterData2019 and identifies relationships between job failures, user behavior, and inefficient resource management [5]. The Azure Waste Case Study also examines production workloads and identifies widespread resource underutilization across Azure infrastructure [6]. Together, these studies show that resource inefficiencies can be identified and measured using historical workload traces.

The QoS-Based Pricing and Scheduling study further explains how uncertainty in resource requirements encourages users to over-provision to avoid job failures or performance issues [4]. Although the study focuses on provider-side pricing and scheduling, it highlights how resource decisions made before job submission contribute to inefficiency. These findings support TraceAdvisor's approach of providing pre-execution CPU and memory recommendations to reduce unnecessary resource allocation while maintaining workload reliability.

2) Datasets and Trace Foundations: Production workload traces provide historical resource usage data that can be used to understand workload behavior and support future resource allocation decisions. Resource Central, a large-scale Microsoft study of Azure production VM workloads, demonstrates that historical workload behavior can support resource prediction and management [2]. This supports TraceAdvisor's use of historical execution data to generate resource recommendations before job submission.

The primary dataset used in TraceAdvisor is Google ClusterData2019 [7], which contains job and task lifecycle events, resource requests, and observed resource usage from production clusters. A subset of this dataset is used for processing and experimentation [8]. These records allow TraceAdvisor to reconstruct workload executions, identify recurring jobs, and analyze CPU and memory usage patterns to support resource recommendations.

Alibaba GPU 2020 serves as a second production dataset, extending the evaluation to recurring machine learning workloads. Weng et al. [9] examined Alibaba's Platform for Artificial Intelligence (PAI) workload traces and highlighted resource management challenges, such as low GPU utilization, varying workload demands, and CPU bottlenecks. TraceAdvisor uses Alibaba's recurring workload information and available CPU and memory measurements to evaluate resource recommendations in a different production environment.

Other production trace studies provide further evidence of resource allocation inefficiencies. Azure VM traces provide additional production workload data [10], while the Azure

Waste Case Study identifies widespread resource underutilization [6]. The Philly GPU Cluster study also identifies GPU underutilization and scheduling challenges in deep learning workloads [11]. Splitwise and ModServe also demonstrate that AI workloads have different resource demands across processing stages [12], [13]. These studies highlight the importance of understanding workload behavior when making resource allocation decisions.

Together, these studies show that historical workload traces can provide useful information for identifying recurring patterns, measuring resource usage, and improving resource allocation decisions. By using Google ClusterData2019 and Alibaba GPU 2020, TraceAdvisor applies historical workload profiling and pre-execution resource recommendation methods to two different production workload environments.

3) Existing Resource Recommendation Approaches: Existing resource recommendation approaches focus on correcting user resource estimates, optimizing configurations for recurring workloads, and using historical data to improve resource management. Although these approaches demonstrate the value of historical workload information, many operate at runtime, require scheduler integration, or rely on online experimentation. TraceAdvisor builds on these findings by providing offline CPU and memory recommendations before job submission.

Early research on runtime prediction shows that user-provided estimates are often inaccurate and that historical predictions can improve scheduling performance [4], [14]. However, these approaches primarily focus on runtime estimation within existing schedulers rather than CPU and memory recommendations before execution.

Configuration optimization studies such as Accordia, CherryPick, and Scavenger use historical workload behavior and optimization techniques to identify suitable cloud configurations [3], [15], [16]. Accordia focuses on recurring batch workloads, while Scavenger extends configuration optimization to machine learning workloads. However, these approaches rely on configuration search or online experimentation, whereas TraceAdvisor uses historical execution data to generate resource recommendations without requiring additional workload executions.

Other studies demonstrate the value of historical resource usage and percentile-based methods. Resource Central uses historical Azure VM workload data to support resource management decisions [2], while Flex identifies persistent differences between requested and actual resource usage in Google production clusters [1]. Azure downsizing research also uses percentile-based CPU estimates to guide resource allocation decisions [17]. Similarly, TailGuard demonstrates the importance of high-percentile objectives, such as P95 and P99, for managing rare performance events [18]. These findings support TraceAdvisor's use of historical resource profiles and percentile-based safeguards.

Together, these studies show that historical workload data can support resource prediction and improve resource allocation. However, existing approaches often focus on runtime scheduling, configuration optimization, or provider-side resource management. TraceAdvisor differs by combining historical profiling, statistical baselines, and machine learning predictions into an offline framework that provides CPU and memory recommendations before job submission.

4) Research Gap: The literature establishes three consistent findings: (1) over-provisioning and slack are widespread across major cloud platforms such as Google and Azure; (2) production traces contain historical resource usage data, and recurring workloads often show predictable patterns across executions; and (3) offline trace-driven evaluation provides a practical way to study resource allocation at scale. These findings demonstrate that historical workload behavior can support resource allocation decisions and help identify inefficiencies in production environments.

However, a gap remains in existing resource management approaches. Many systems operate at runtime, require modifications to existing schedulers, depend on online experimentation, or focus primarily on configuration optimization. Although these approaches demonstrate that historical data and percentile-based methods can improve resource management, they do not directly address the need for lightweight, offline resource recommendations before job submission. In particular, the reviewed literature does not establish an approach that combines historical profiling of recurring workloads, CPU and memory recommendations, and percentile-based safeguards without requiring scheduler modifications or online profiling.

TraceAdvisor addresses this gap by using historical workload traces to identify recurring jobs, analyze resource usage patterns, and generate risk-aware CPU and memory recommendations before execution. The framework uses historical percentiles to account for higher resource demands while aiming to reduce unnecessary resource allocation. By operating independently of existing schedulers, TraceAdvisor provides a way to evaluate pre-execution resource recommendations across different production workload environments without requiring changes to the scheduling system.

D. Project Objectives and Scope

The primary objective of TraceAdvisor is to develop an offline, trace-driven decision support system that generates pre-execution CPU and memory resource recommendations for recurring cluster workloads. The system uses historical execution traces to identify recurring jobs, analyze resource usage patterns, and generate risk-aware recommendations. The goal is to reduce over-provisioning and improve resource efficiency while maintaining a low risk of under-provisioning to ensure workload reliability.

TraceAdvisor processes historical workload records into structured datasets containing resource requests, observed resource usage, execution duration, and recurring workload identifiers. Historical statistics, including mean usage, variability, and percentile-based estimates (P90, P95, P99), are used to generate resource recommendations. The project also aims to evaluate the effectiveness of these resource recommendations through chronological trace replay. Performance is measured using violation rate, coverage, slack, waste, and utilization metrics to assess the balance between resource efficiency and workload reliability. The project uses Google ClusterData2019

as the primary reference dataset and Alibaba GPU 2020 as a second production workload dataset. Google ClusterData2019 supports historical profiling and peak CPU and memory resource recommendations for recurring workloads. Alibaba GPU 2020 extends the framework to recurring machine learning workloads, using available CPU and memory measurements to evaluate resource recommendations in a different cluster environment. Since the datasets differ in workload structure and resource measurements, the framework uses dataset-specific processing and evaluation methods. TraceAdvisor operates as a non-intrusive advisory system using offline historical data and does not require modifications to existing schedulers.

E. Project Contributions

TraceAdvisor contributes a trace-driven framework that provides pre-execution CPU and memory resource recommendations for recurring cluster workloads. By using historical workload behavior, the framework aims to reduce unnecessary resource allocation while accounting for the risk of under-provisioning. The project combines resource recommendation methods within a reproducible offline evaluation workflow to balance resource efficiency and workload reliability.

A primary contribution is the development of a workload processing and profiling pipeline using Google ClusterData2019. Raw trace records are transformed into structured execution-level summaries containing resource requests, observed CPU and memory usage, execution duration, and recurring workload identifiers. Historical statistics, including mean usage, variability, and high-percentile resource demand (P90, P95, P99), are computed to characterize workload behavior. These profiles support the comparison of original user-requested resources (As-is), historical baselines, machine learning predictions, and Hybrid recommendation policies that combine learned predictions with historical safeguards.

TraceAdvisor also extends the original Google ClusterData2019 pipeline by incorporating the Alibaba GPU 2020 production workload trace. A dataset-specific adapter was developed to transform Alibaba's job, task, instance, and resource monitoring records into a standardized format while preserving differences in resource measurements. The integration includes workload eligibility filtering, handling of missing data, validation of source and output records, and reproducible dataset generation. The adapter was validated using the full Alibaba dataset, providing a foundation for applying historical profiling and resource recommendation methods to a second production workload environment.

Finally, TraceAdvisor includes a time-aware evaluation framework that compares resource recommendations against observed workload demand. Chronological training, validation, and test splits are used to preserve temporal consistency and prevent data leakage. Recommendation performance is measured using violation rate, coverage, slack, waste, and utilization to evaluate the trade-off between resource efficiency and workload reliability. The evaluation also accounts for differences in CPU and memory measurements across datasets, allowing each workload environment to be evaluated using its available resource information. Together, these contributions establish a reproducible framework for generating and evaluating pre-execution resource recommendations across different production workloads.

II. Project Architecture

A. Introduction and System Overview

B. Architecture Subsystems

1) Data Ingestion and Dataset Adapters:
2) Execution Reconstruction and Occurrence Modeling:
3) Historical Features and Time-Ordered Data:
4) Recommendation Engine:
5) Shared Metrics and Evaluation Framework:
6) Artifact Repository and Presentation Layer:

III. Technology Descriptions

A. Data Processing and Analysis Stack

B. Machine Learning Libraries and Models

C. Dataset and Trace Sources

1) Primary Dataset: Google ClusterData 2019:
2) Validation Datasets: Azure Resource Central, Alibaba, ModServe, and Splitwise:

D. Dataset Adapters and Unit Normalization

E. Artifact Storage, Reproducibility, and Presentation Technologies

IV. Project Design

A. Data Pipeline Design

B. Recommendation Model Design

1) Historical Percentile Baselines:
2) Machine Learning Forecasting:
3) Hybrid Risk-Aware Layer:

C. Replay and Evaluation Framework Design

D. Artifact and Presentation Layer Design

V. Project Implementation

A. Data Pipeline

1) Ingestion, Unit Normalization, and Dataset Adapters:
2) Execution Reconstruction and Dataset-Specific Temporal Features:
3) Peak Usage and Resource Metrics:

B. Baseline Recommenders

1) Request-As-Is:
2) Mean and Percentile Recommendations:
3) Cold-Start Handling and Fallbacks:

C. Machine Learning Recommender

1) Feature and Target Preparation:
2) Model Training and Selection:
3) Validation-Based Model Comparison:

D. Hybrid Risk-Aware Recommender

E. Shared Metrics and Evaluation Framework

F. Artifact Storage and Developer-Facing Outputs

VI. Testing and Verification

A. Data and Pipeline Correctness Checks

B. Temporal Leakage and Split Integrity

C. Model and Baseline Validation

D. Replay Evaluation Metrics

1) Violation and Utilization:
2) Resource Waste:
3) Valid-Row Denominators and Missing-Value Handling:

VII. Performance, Results, and Discussion

A. Google ClusterData 2019 Results

1) Baseline Recommendation Results:
2) ML and Hybrid Recommendation Results:
3) Feature Importance and Explainability:

B. Cross-Platform Validation Results

C. Discussion of Findings

VIII. Operations, Reproducibility, and Future Deployment

A. Offline Artifact Workflow

B. Reproducibility and Repository Layout

C. Interactive Developer Dashboard (Planned)

IX. Summary, Conclusions, and Recommendations

A. Summary

B. Conclusions

C. Limitations and Recommendations for Further Research

References

[1] T. N. Le and Z. Liu, "Flex: Closing the gaps between usage and allocation," arXiv preprint, arXiv:2006.01354, Jun. 2020. doi: 10.48550/arXiv.2006.01354.

[2] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms," in Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China: ACM, 2017, pp. 153–167. doi: 10.1145/3132747.3132772.

[3] Y. Liu, H. Xu, and W. C. Lau, "Cloud configuration optimization for recurring batch-processing applications," IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 5, pp. 1495–1507, May 2023. doi: 10.1109/TPDS.2023.3246086.

[4] T. Sandholm, J. Ward, F. Balestrieri, and B. A. Huberman, "QoS-based pricing and scheduling of batch jobs in OpenStack clouds," arXiv preprint, arXiv:1504.07283, 2015. doi: 10.48550/arXiv.1504.07283.

[5] F. H. Bappy, T. Islam, T. S. Zaman, R. Hasan, and C. Caicedo, "A deep dive into the Google cluster workload traces: Analyzing the application failure characteristics and user behaviors," in Proc. 10th International Conference on Future Internet of Things and Cloud (FiCloud), 2023, pp. 103–108. doi: 10.1109/FiCloud58648.2023.00023.

[6] B. Everman, M. Gao, and Z. Zong, "Evaluating and reducing cloud waste and cost—A data-driven case study from Azure workloads," Sustainable Computing: Informatics and Systems, vol. 35, p. 100708, 2022. doi: 10.1016/j.suscom.2022.100708.

[7] J. Wilkes et al., "Google Borg cluster traces (ClusterData2019)," 2019. [Online]. Available: https://github.com/google/cluster-data. Accessed: Feb. 2026.

[8] D. Mwiti, "Google 2019 Cluster Sample," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/derrickmwiti/google-2019-cluster-sample. Accessed: Apr. 2026.

[9] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding, "MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in Proc. 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2022, pp. 945–960. [Online]. Available: https://www.usenix.org/conference/nsdi22/presentation/weng.

[10] Microsoft, "Microsoft Azure Traces," GitHub repository. [Online]. Available: https://github.com/Azure/AzurePublicDataset. Accessed: Apr. 2026.

[11] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of large-scale multi-tenant GPU clusters for DNN training workloads," in Proc. USENIX Annual Technical Conference (ATC), 2019. [Online]. Available: https://www.usenix.org/system/files/atc19-jeon.pdf.

[12] P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," arXiv preprint, arXiv:2311.18677, May 2024. doi: 10.48550/arXiv.2311.18677.

[13] H. Qiu et al., "ModServe: Modality- and stage-aware resource disaggregation for scalable multimodal model serving," arXiv preprint, arXiv:2502.00937, Oct. 2025. doi: 10.48550/arXiv.2502.00937.

[14] D. Scheinert, A. Alamgiralem, J. Bader, J. Will, T. Wittkopp, and L. Thamsen, "On the potential of execution traces for batch processing workload optimization in public clouds," in 2021 IEEE International Conference on Big Data (Big Data), 2021, pp. 3113–3118. doi: 10.1109/BigData52589.2021.9671275.

[15] O. Alipourfard and M. Yu, "CherryPick: Adaptively unearthing the best cloud configurations for big data analytics," in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2017. [Online]. Available: https://www.usenix.org/system/files/conference/nsdi17/nsdi17-alipourfard.pdf.

[16] S. Tyagi and P. Sharma, "Scavenger: A cloud service for optimizing cost and performance of ML training," in 2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid), May 2023, pp. 403–413. doi: 10.1109/CCGrid57682.2023.00045.

[17] N. Huang, A. Li, S. Zhang, and Z. Zong, "Reducing cloud expenditures and carbon emissions via virtual machine migration and downsizing," in 2023 IEEE International Performance, Computing, and Communications Conference (IPCCC), Nov. 2023, pp. 74–81. doi: 10.1109/IPCCC59175.2023.10253871.

[18] Z. Wang, H. Li, L. Sun, S. Rosenkrantz, H. Che, and H. Jiang, "A tail latency SLO guaranteed task scheduling scheme for user-facing services," IEEE Transactions on Parallel and Distributed Systems, vol. 36, no. 4, pp. 759–774, Apr. 2025. doi: 10.1109/TPDS.2025.3542638.